

SCIOLY.APP — MASTER PRODUCT REQUIREMENT DOCUMENT & SYSTEM ARCHITECTURE

1. EXECUTIVE SUMMARY & SYSTEM PHILOSOPHIES

1.1 Product Vision

sciolly.app is an aggressive, high-density operations and practice platform ("The Bloomberg Terminal for Science Olympiad") built for high school and middle school competitors, team captains, and coaches. It replaces fragmented PDFs, wikis, and static test banks with an algorithmic "Living Engine" of practice, team logistics, and structured study.

1.2 Non-Negotiable Engineering Principles

- Zero User-Facing AI Latency:** No live LLM calls exist in the student practice path. Students interact exclusively with pre-generated, human-reviewed static question banks. All user-side operations execute under <100ms.
- Sub-100ms Optimistic UI:** The client application (React SPA) treats browser memory as the source of truth, updating UI instantaneously and syncing back to Supabase asynchronously.
- High Information Density & Cyberpunk Aesthetic:** Dark mode background (#050505), glassmorphism containers (rgba(255,255,255,0.05)), neon cyan (#00f0ff) and warning red (#ff2a2a) accents, high-contrast monospace typography for data points.
- Offline-First AI Pipeline:** AI inference is strictly isolated to the Platform Admin Factory (running overnight batch generation via NVIDIA NIM DeepSeek V4) and the Ephemeral Binder Audit (NVIDIA Nemotron Parse/VL).

2. TECH STACK & INFRASTRUCTURE ARCHITECTURE

Layer	Technology	Operational Justification
Frontend Framework	React 18 + Vite + TypeScript	Instant HMR, strict type-checking across complex state models.
UI & Styling	Tailwind CSS v3 + Radix UI Primitives	Zero runtime CSS overhead, fully accessible unstyled primitives custom-styled to cyberpunk HUD aesthetics.
Global UI State	Zustand	Lightweight client state for timers, combos, active UI overlays without context re-render overhead.
Data Fetching & Cache	TanStack Query (React Query v5)	Stale-While-Revalidate caching pattern for instant tab navigation and zero loading spinners.
Backend / DB / Auth	Supabase (PostgreSQL 15 + RLS)	Relational database with built-in Row-Level Security, Auth JWTs, Storage Buckets, and RPC functions.

AI Batch Generation	NVIDIA NIM API (DeepSeek V4 Flash/Pro)	Free/High-throughput 40 RPM API for admin batch question synthesis.
AI Vision Audit	NVIDIA NIM API (Nemotron Parse / VLM)	Multi-modal vision parsing for Binder PDF analysis.

3. MASTER POSTGRESQL SCHEMA (SUPABASE DDL)

```
-- =====
-- ENUMS
-- =====
CREATE TYPE division_type AS ENUM ('B', 'C');
CREATE TYPE team_role AS ENUM ('coach', 'captain', 'officer', 'member');
CREATE TYPE question_status AS ENUM ('draft', 'live', 'archived');
CREATE TYPE question_type AS ENUM ('mcq', 'diagram', 'calc');

-- =====
-- USERS & PROFILES
-- =====
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  handle TEXT UNIQUE NOT NULL,
  division division_type NOT NULL DEFAULT 'C',
  platform_role TEXT NOT NULL DEFAULT 'user', -- 'admin' for founder
  avatar_id TEXT NOT NULL DEFAULT 'recruit_helmet',
  team_id UUID NULL, -- Linked to teams(id)
  xp INT NOT NULL DEFAULT 0,
  rank_title TEXT NOT NULL DEFAULT 'Recruit Level 1',
  current_streak INT NOT NULL DEFAULT 0,
  questions_answered INT NOT NULL DEFAULT 0,
  visibility TEXT NOT NULL DEFAULT 'public', -- 'public' | 'private'
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- TEAMS
-- =====
CREATE TABLE teams (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  school_name TEXT NOT NULL,
  division division_type NOT NULL DEFAULT 'C',
  join_code_student VARCHAR(6) UNIQUE NOT NULL,
  join_code_admin VARCHAR(6) UNIQUE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE team_roster (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  role team_role NOT NULL DEFAULT 'member',
  joined_at TIMESTAMPTZ DEFAULT NOW(),
```

```

        UNIQUE(team_id, user_id)
    );

ALTER TABLE profiles ADD CONSTRAINT fk_user_team FOREIGN KEY (team_id) REFERENC

-- =====
-- TAXONOMY (EVENTS & TOPICS)
-- =====
CREATE TABLE taxonomy_events (
    id TEXT PRIMARY KEY, -- e.g., 'anatomy'
    name TEXT NOT NULL,
    division division_type NOT NULL DEFAULT 'C',
    active BOOLEAN DEFAULT TRUE
);

CREATE TABLE taxonomy_topics (
    id TEXT PRIMARY KEY, -- e.g., 'anatomy_endocrine'
    event_id TEXT REFERENCES taxonomy_events(id) ON DELETE CASCADE,
    name TEXT NOT NULL
);

-- =====
-- QUESTION BANK
-- =====
CREATE TABLE questions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    season_ids INT[] NOT NULL DEFAULT '{2026}',
    division division_type NOT NULL DEFAULT 'C',
    event_id TEXT REFERENCES taxonomy_events(id) ON DELETE CASCADE,
    topic_id TEXT REFERENCES taxonomy_topics(id) ON DELETE CASCADE,
    sub_topic TEXT NOT NULL, -- Used for Clinic intervention grouping
    question_type question_type NOT NULL DEFAULT 'mcq',
    status question_status NOT NULL DEFAULT 'draft',

    stem TEXT NOT NULL,
    media_url TEXT NULL,
    options JSONB NOT NULL, -- {"a": "...", "b": "...", "c": "...", "d": "..."}
    correct_key VARCHAR(1) NOT NULL,
    explanation TEXT NOT NULL,
    citation TEXT NOT NULL,

    times_served INT DEFAULT 0,
    global_accuracy FLOAT DEFAULT 0.0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- USER PROGRESSION & QUEUES
-- =====
CREATE TABLE user_weakness_map (
    user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    topic_id TEXT REFERENCES taxonomy_topics(id) ON DELETE CASCADE,
    total_attempts INT DEFAULT 0,
    correct_attempts INT DEFAULT 0,
    PRIMARY KEY (user_id, topic_id)
);

```

```

CREATE TABLE user_history (
    user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    question_id UUID REFERENCES questions(id) ON DELETE CASCADE,
    answered_at TIMESTAMPTZ DEFAULT NOW(),
    is_correct BOOLEAN NOT NULL,
    PRIMARY KEY (user_id, question_id, answered_at)
);

CREATE TABLE reinjection_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    question_id UUID REFERENCES questions(id) ON DELETE CASCADE,
    unlock_at TIMESTAMPTZ NOT NULL,
    resolved BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- TEAM PORTAL (CMD)
-- =====
CREATE TABLE team_missions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
    created_by UUID REFERENCES profiles(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    target_event_id TEXT REFERENCES taxonomy_events(id),
    goal_type TEXT NOT NULL, -- 'volume', 'timed_test', 'accuracy'
    target_value INT NOT NULL,
    deadline TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE user_mission_progress (
    mission_id UUID REFERENCES team_missions(id) ON DELETE CASCADE,
    user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    current_value INT DEFAULT 0,
    completed BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (mission_id, user_id)
);

CREATE TABLE team_posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
    author_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    content TEXT NOT NULL,
    is_pinned BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE team_vault_resources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
    event_id TEXT REFERENCES taxonomy_events(id),
    title TEXT NOT NULL,
    url TEXT NOT NULL,

```

```

category TEXT NOT NULL, -- 'Google Drive', 'Quizlet', 'YouTube', 'Document'
created_by UUID REFERENCES profiles(id),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- BINDER GENESIS ASSETS
-- =====
CREATE TABLE binder_assets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id TEXT REFERENCES taxonomy_events(id) ON DELETE CASCADE,
  type TEXT NOT NULL, -- 'table' | 'diagram' | 'formula' | 'glossary'
  name TEXT NOT NULL,
  payload_html TEXT NOT NULL,
  preview_url TEXT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

Row-Level Security (RLS) Policies

```

ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE questions ENABLE ROW LEVEL SECURITY;
ALTER TABLE teams ENABLE ROW LEVEL SECURITY;
ALTER TABLE team_roster ENABLE ROW LEVEL SECURITY;

-- Profiles: Anyone can view public profiles; only owner updates.
CREATE POLICY "Public Profiles View" ON profiles FOR SELECT USING (true);
CREATE POLICY "Profile Self Update" ON profiles FOR UPDATE USING (auth.uid() =

-- Questions: Live questions readable by authenticated users; write restricted
CREATE POLICY "Read Live Questions" ON questions FOR SELECT USING (status = 'li
CREATE POLICY "Admin Questions Write" ON questions FOR ALL USING (
  EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND platform_role = 'a
);

-- Teams: Members can read team data.
CREATE POLICY "Team Members Read" ON teams FOR SELECT USING (
  EXISTS (SELECT 1 FROM team_roster WHERE team_id = teams.id AND user_id = au
);

```

4. AUTHENTICATION, ONBOARDING & PERMISSIONS

4.1 Onboarding & Verification Protocol

1. **Signup:** Authentication via Google OAuth or Email/Password via Supabase Auth.
2. **Setup Wizard** (/setup):

- **Step 1 (Handle):** Unique text handle input `@handle` . Validated via live debounced check against profiles .
- **Step 2 (Classification):** Toggle Division [Division B] or [Division C] . Locks question pools accordingly.
- **Step 3 (Deployment):** Enter 6-digit Team Join Code OR proceed as "Freelancer" (`team_id: null`).

3. Team Creation:

- User clicks "Create Team" in settings. Generates two 6-character unique codes:
 - `join_code_admin` (Secret: for Coaches/Advisors).
 - `join_code_student` (Public: for Students).

4.2 Permission Hierarchy Matrix

Role	Manage Roster / Roles	Create Missions	Post Updates	Edit Vault	View Private Stats
Platform Admin <i>(Founder)</i>	✔ (Global)	N/A	N/A	N/A	✔ (Global)
Coach (Owner)	✔ (Team)	✔	✔	✔	✔ (Team)
Captain (Prez)	✘	✔	✔	✔	✔ (Team)
Officer (Board)	✘	✘	✔	✔	✘
Member	✘	✘	✘	✘	✘

5. DETAILED FEATURE SPECIFICATIONS

5.1 OPS: Casual Mode ("The Arena" & "The Clinic")

A. Setup Lobby

- **Event Selector:** Searchable grid displaying active events matching user's division.
- **Topic Mixer:** Checkboxes allowing users to exclude specific broad topics (e.g., exclude "Nervous System").
- **CTA:** Massive Neon Cyan button [ENTER FLOW] .

B. The Serving Algorithm ("The Brain")

When `getNextQuestion()` is called, the client runs the following deterministic order:

1. **Spaced Repetition Queue Check:** Query `reinjection_queue` WHERE `user_id = current_user` AND `unlock_at <= NOW()` AND `resolved = false` . If items exist, serve highest priority.

2. **Weakness Bias Query:** Fetch topic with lowest accuracy in `user_weakness_map` . Apply a **2.0x sampling weight** to questions belonging to this topic ID in the active pool.
3. **New Content Randomization:** Fetch random question WHERE `event_id = selected_event` AND `id NOT IN (user_history)` AND `status = 'live'` .

C. Arena Interface & HUD

- **Header HUD:** Event/Topic path tag, source citation (e.g., *Campbell Biology Ch. 17*), active Combo Counter (`< x5`), and [`X End Session`] escape button.
- **Question Card:** Stem display with optional high-res diagram slot, 4 clickable option rectangles (A) through (D) .
- **Feedback Loop:**
 - **Correct:** Border flashes Neon Green (`#00ff66`), plays mechanical snap sound. Combo increments. Next button auto-focuses for Enter/Space press.
 - **Incorrect:** Border flashes Red (`#ff2a2a`), plays error thud. Option explanation auto-expands. **Next button is disabled for 3.0 seconds** with visual countdown ring to prevent skipping explanations.

D. The Clinic ("Surgical Intervention")

- **Trigger:** Temporary session counter `session_streak_incorrect` tracks wrong answers per `sub_topic` . If user gets **3 questions wrong on the exact same sub-topic** within a session:
- **Interruption:** Session locks. UI blurs. Red banner appears: [ `REPAIR SYSTEM REQUIRED`] .
- **3-Step Modal Flow:**
 1. **READ:** Static concise concept summary card.
 2. **SEE:** Static annotated diagram or equation block.
 3. **DO:** Simplified 2-option verification question.
- **Resolution:** Passing closes modal, grants `+50 XP Repair Bonus` , and resets `session_streak_incorrect` to 0.

E. Session Commit & Reinjection

Upon clicking `End Session` :

- Single RPC call `submit_casual_session` updates total user XP, increments `questions_answered` , recalculates `user_weakness_map` .
- Any question answered incorrectly is inserted into `reinjection_queue` with `unlock_at = NOW() + INTERVAL '72 hours'` .

5.2 OPS: Timed Practice ("The Black Box")

A. Configuration

- Select Event, select Duration (10m Sprint / 15 Qs or 50m Standard / 50 Qs), select Density (Standard 1.0 min/Q or Blitz 45 sec/Q).

B. Exam Stage & HUD

- **HUD:** Digital countdown timer (turns pulsing red under 5:00), topic title, Question Navigator Grid on sidebar (shows Unanswered, Answered, Flagged).
- **Constraints:** No immediate answer feedback, no explanation views, right-click/copy disabled.
- **Crash Protection:** Every selection writes instantly to `localStorage.setItem('active_timed_session', state)` . On page refresh or network drop, exam restores state and remaining time instantly.
- **Anti-Cheat Monitor:** Listens to `visibilitychange` . If user switches tabs, `document.hidden` flags session as `untrusted = true` . Untrusted scores generate XP but are excluded from public leaderboards.

C. The Autopsy View

- **Scoreboard:** Raw score (e.g. 42/50), Accuracy %, XP payout ($\text{Base } 100 + \text{Score} * 10$), Percentile ranking vs. global average, Speed per question vs. average.
- **Scantron Interactive Strip:** Horizontal color-coded bar (Green = Correct, Red = Incorrect, Grey = Skipped). Clicking any square expands detailed explanation and user response comparison.

5.3 CMD: Team Operations & Dashboard

A. Freelancer vs. Team View

- **Freelancer (`team_id == null`):** Central dashboard shows Recruitment Callout panel ("Science Olympiad is a Team Sport") with [Enter Team Code] field, personal level progress, and active personal Quests.
- **Team Recruit (`team_id != null`):** Central dashboard renders 4 functional tabs:

B. Tab 1: Roster & Intel ("The War Room")

- **Coverage Matrix Grid:** Rows = 23 Events; Columns = Varsity / JV slots. Drag-and-drop user avatars into assigned slots. Displays red warning badges on unassigned events.
- **Roster Table:** Sortable by Name, Role, Last Active, Weekly XP. Includes "Nudge" button to notify inactive users.

C. Tab 2: Missions ("Assignments")

- **Mission Creation:** Admin/Captain targets specific users or event squad. Defines task (e.g., "Complete 50 Casual Problems in Astronomy"), deadline, and XP reward.
- **Student Mission Stream:** Status cards marked **OVERDUE** (flashing red), **IN PROGRESS** (percentage bar), or **COMPLETE** (green checkmark). Deep-links directly to target event arena.

D. Tab 3: Comms & Vault

- **Comms Feed:** Chronological announcement stream with pinned coach announcements.
- **The Vault:** Categorized external links repository (Google Drive folders, Quizlet sets, YouTube playlists). **Strictly zero file uploads** are permitted in Vault to prevent storage cost inflation and copyright infringement.

E. Tab 4: Team Leaderboard

- Filters by Timeframe (This Week , This Month , All Time) and Metric (XP , Questions , Accuracy). Highlights active user's row.

5.4 Binder Planner: Genesis & Vision Audit

A. Genesis Mode ("The Blueprint")

- **Sidebar Map:** Chapter breakdown derived from official rules.
- **Workspace:** Page template with drop zones: [INSERT DEFINITION TABLE] , [INSERT FORMULA BLOCK] , [INSERT DIAGRAM] .
- **Toolbox:** Library of pre-formatted HTML/CSS widgets stored in `binder_assets` table. Users drag widgets into drop zones.
- **Clipboard Export:** [COPY TO CLIPBOARD] button serializes active DOM canvas into single HTML string with inline CSS, formatted for direct pasting into Google Docs or Word.

B. Vision Audit Mode ("The Referee")

- **Gatekeeper Rules:** Automated PDF page count check. Auto-rejects files >2 pages for Cheat Sheet events (Microbe Mission, Chem Lab). Rejects uploads for Brain-Only events (Codebusters, Fermi).
- **Processing Flow:**
 1. PDF uploaded to temporary memory buffer.
 2. Page images sent to **NVIDIA NIM Nemotron Parse / VLM API**.
 3. Vision prompt evaluates 4 criteria: Index/TOC presence, Margin size (<0.5"), Font size density (<9pt), and Missing required topic keywords.
- **Report Card:** Displays Overall Score (0-100), Tier Badge (Gold/Silver/Bronze), Green Flags (passed rules), and Red Flags (missing concepts/format errors).
- **Ephemeral Cleanup:** The PDF file is permanently deleted from memory/server immediately after outputting the JSON report.

5.5 Profile, Social & Gamification

A. Leveling & Title Progression

- Level equation: $\text{Level} = \lfloor \sqrt{\text{XP}/100} \rfloor + 1$.
- XP Lookup Titles: Level 1-5 (Recruit), 6-10 (Specialist), 11-20 (Tactician), 21-35 (Master), 36+ (National Legend).

B. Personnel Profile Page

- **Hero Section:** Avatar, handle, rank title, current day streak, total XP.
- **Radar Chart:** 6-axis spider graph evaluating performance across subject categories (Life Science, Physical Science, Inquiry, Earth & Space, Chemistry, Physics).
- **Trophy Case:** Grid of earned and locked badges. Clicking badge displays unlock requirements (e.g., "Answer 500 Astronomy Questions").
- **Public / Private Toggle:** Setting profile to Private hides radar chart, stats, and search index from non-team members.

5.6 Platform Admin Console (/admin)

A. Access Control

- Direct route /admin gated by Supabase Auth check: `profiles.platform_role === 'admin'` . Unauthorized users receive instant 404 redirect.

B. AI Batch Generation Factory

- **NVIDIA NIM Configuration:** Select Event, Division (B / C), Topic, Sub-Topic, and Batch Count (10-50 questions).
- **System Prompt:** Hardcoded strict JSON generator prompt requesting stem, 4 options, correct key, explanation, and textbook citation.
- **Execution:** Issues HTTP POST to `https://integrate.api.nvidia.com/v1/chat/completions` using DeepSeek V4 API key. Parses JSON array response into `questions` table with `status = 'draft'` .

C. Review Queue ("Tinder for Science")

- Single-card review UI displaying draft question, options, correct answer highlighted in green, explanation, and tags.
- **Keyboard Hotkeys:**
 - A (Approve): Set status to `'live'` , push to main DB, load next card.
 - R (Reject): Hard delete draft record from DB, load next card.
 - E (Edit): Convert card text into editable form inputs to fix typos/hallucinations, then save.
 - S (Skip): Send draft card to back of review queue.

D. Warehouse & Manual Override

- **Season Migration Tool:** Select Season + Event + Topic, batch update `season_ids` array (e.g. `append 2027`), or set status to `'archived'` .
- **Regex Raw Text Ingester:** Large text area for pasting raw test questions. Client-side regex parses `"A)"` , `"B)"` , `"Ans: "` tags and auto-fills draft form fields.

6. INTEGRATION ARCHITECTURE & API FLOWS

6.1 NVIDIA NIM Batch Question Generator Integration

```
// Edge / Admin Serverless Function: generateBatch.ts
import { createClient } from '@supabase/supabase-js';

export async function generateNvidiaBatch(config: {
  event_id: string;
  topic_id: string;
  sub_topic: string;
  division: 'B' | 'C';
  count: number;
}) {
  const systemPrompt = `You are an elite Science Olympiad National Exam Supervisor.
  Generate exactly ${config.count} multiple choice questions for Division ${config.division}.
  Output MUST be a valid JSON array of objects matching this exact schema:
  [
    {
      "stem": "Question text here",
      "options": {"a": "Option A", "b": "Option B", "c": "Option C", "d": "Option D"},
      "correct_key": "a",
      "explanation": "Detailed explanation of concept",
      "citation": "Standard Textbook Reference"
    }
  ]
  Output ONLY raw JSON. Do not include markdown formatting or extra text.`;

  const response = await fetch('https://integrate.api.nvidia.com/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.NVIDIA_NIM_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      model: 'deepseek-ai/deepseek-r1', // or deepseek-v4 flash
      messages: [{ role: 'system', content: systemPrompt }],
      temperature: 0.2,
      max_tokens: 4096
    })
  });

  const data = await response.json();
  const rawContent = data.choices[0].message.content.trim();
  const questions = JSON.parse(rawContent.replace(/```json|```/g, ''));

  // Insert into Supabase as drafts
  const rows = questions.map((q: any) => ({
    ...q,
    division: config.division,
    event_id: config.event_id,
    topic_id: config.topic_id,
    sub_topic: config.sub_topic,
    status: 'draft'
  }));

  const supabase = createClient(process.env.SUPABASE_URL!, process.env.SUPABASE_SECRET_KEY!);
  const { data: inserted, error } = await supabase.from('questions').insert(rows);
  if (error) console.error('Error inserting questions:', error);
  return inserted;
}
```

```
const { error } = await supabase.from('questions').insert(rows);  
if (error) throw error;  
return { success: true, count: rows.length };  
}
```

7. SECURITY, COMPLIANCE & PERFORMANCE BENCHMARKS

7.1 Data Privacy & Protection

- **COPPA/FERPA Compliance:** Minimal data collection. No real name, phone number, or address required.
- **No UGC PDF Hosting:** Team Vault strictly accepts URL strings. No PDF uploads are stored on servers except during transient Binder Audits.
- **Ephemeral Processing Guarantee:** Uploaded Binder PDFs during Audit are processed entirely in memory buffers and immediately purged after JSON generation.

7.2 Anti-Cheat & System Integrity

- **XP Validation:** Server RPC function checks submitted time vs. question volume ($\text{max_possible_xp} = \text{count} * 15$). Rejects submissions exceeding physical threshold.
- **Rate Limiting:** API requests capped via Supabase Edge Function rate-limiting logic to prevent scraping.

7.3 Performance Targets

- **Initial Page Load:** < 1.2s on standard mobile connections.
- **Practice Question Transition:** < 50ms local UI response.
- **Database Query Response:** < 80ms via indexed Postgres queries.

